Day 4: Embeddings, Semantic Search and Vector Databases

Summary and interview preparation · Sagar DS · November 2026

Part 1: What we learnt

1. Keyword search vs meaning search Keyword search only finds documents that contain the **same words** as the question. Meaning (semantic) search finds documents with the **same meaning**, even when the words are different. For example, "server is out of space" can find a document titled "Disk full".

2. Embeddings An embedding turns a piece of text into a **long list of numbers** (for example 768) that captures its meaning. Texts with similar meanings get similar numbers. A whole sentence or paragraph becomes **one** list, whatever its length.

3. Embedding model vs chat model

	Embedding model	Chat model
Input	Text	Text
Output	Numbers (a vector)	Text (an answer)
Job	**Find**: represent meaning for search	**Write**: answer, explain, summarise

They are different tools, often used together.

4. Similarity and distance Two embeddings are compared with a score.

- **Similarity:** higher = closer in meaning.
- **Distance:** lower = closer in meaning.

Scores should be compared **against each other**; one number on its own means little.

5. Chunking Long documents are cut into smaller pieces (chunks) so each piece has a clear, focused meaning, and search can return exactly the relevant part. Adding the document's title to each chunk keeps its context. Chunk size is a trade-off: too big blurs the meaning, too small loses context.

6. Vector database For every chunk, a vector database stores:

- **The text**, to read
- **Its embedding**, to search
- **Metadata** such as the source document, to cite and filter

The embeddings come from **your own documents**, created with a **pre-trained** embedding model.

7. How a search works

1. The question is turned into an embedding, using the **same** model as the documents.

2. It is compared with all stored embeddings.
3. The **closest few** (the "top-k") are returned, with their original text and source.

8. Search is not perfect, so test it In our tests, 4 of 5 questions found the right document with no shared keywords. Misses came from **ambiguous questions** and **generic sections** (such as escalation steps that look similar everywhere). Retrieval quality must be measured, and it improves with clearer, more specific documents.

9. Where this leads: RAG This is the **"R" (Retrieval)** in **RAG** (Retrieval-Augmented Generation). Next, the retrieved text is handed to a chat model so it can **answer from your documents**, with sources.

One line to remember: *Embeddings turn meaning into numbers; a vector database finds the closest meanings; the chat model writes the answer.*

Part 2: Interview questions and answers

Basics

Q1. What is an embedding? An embedding is a list of numbers (a vector) that represents the meaning of a piece of text. Texts with similar meanings get similar vectors, so they can be compared mathematically.

Q2. What is semantic search, and how is it different from keyword search? Keyword search matches exact words. Semantic search matches meaning, using embeddings. "Server is out of space" shares no words with "Disk full", so keyword search misses it, but semantic search finds it.

Q3. What is the difference between an embedding model and a chat model? An embedding model turns text into a vector for search and comparison. A chat model turns text into text: it generates answers. In RAG, the embedding model retrieves the right information and the chat model writes the answer from it.

Q4. What does "dimensions" mean for an embedding? The number of values in the vector, for example 384, 768 or 1,536. More dimensions can capture finer shades of meaning, but take more storage and computation.

Similarity and search

Q5. What is cosine similarity? A measure of how similar two vectors are, based on the angle between them. Close to 1 means very similar meaning; close to 0 means unrelated. It is the most common way to compare text embeddings.

Q6. What is the difference between similarity and distance? They are two views of the same idea. Similarity: higher means closer. Distance: lower means closer. Cosine distance is simply 1 minus cosine similarity. Vector databases often report distance.

Q7. What is "top-k" retrieval? Returning the k closest results to a query, for example the top 3. Returning several results gives the chat model more context, and makes the search more forgiving if the best match is not ranked first.

Q8. Why must the query and the documents use the same embedding model? Each model creates its own "map" of meaning. Vectors from different models are not comparable, just as coordinates from two different maps cannot be compared. Changing the model means re-embedding all documents.

Vector databases

Q9. What is a vector database? A database designed to store embeddings and quickly find the ones closest to a query vector. Examples: ChromaDB, Pinecone, Weaviate, Milvus, Qdrant, and pgvector (a PostgreSQL extension).

Q10. What does a vector database store for each item? Three things: the original text, its embedding, and metadata (such as source, section or date), plus a unique ID. The embedding is used to search, the text is returned, and the metadata is used for citations and filtering.

Q11. How is a vector database different from a normal database? A normal database answers exact questions ("where host = web01") and is good at lookups, filters and totals. A vector database answers "what is most similar to this?" and is good at similarity search. Many systems use both together.

Q12. How do vector databases stay fast with millions of items? Instead of comparing the query with every stored vector, they use approximate nearest neighbour (ANN) indexes, such as HNSW, which find very close matches quickly by searching only a small part of the data. They trade a tiny amount of accuracy for a large gain in speed.

Chunking

Q13. Why do we split documents into chunks? One embedding for a whole document blurs its meaning into an average, so search becomes imprecise. Chunks keep each embedding focused on one topic, so search returns the relevant section. Chunks also keep the text sent to the LLM small and within its context window.

Q14. How do you choose a chunk size? It is a trade-off. Too big: meaning is blurred and tokens are wasted. Too small: chunks lose context and answers get split. Common approaches are splitting by headings or paragraphs, or by a fixed size with a small overlap, then testing which retrieves best on real questions.

Q15. What is chunk overlap, and why use it? Repeating a small part of the previous chunk at the start of the next one. It prevents an important sentence from being cut in half between two chunks and losing its meaning.

Q16. Why add the document title to each chunk? A section such as "Fix: compress old logs" does not say which problem it fixes. Adding the title ("Disk full – Fix") keeps the context, so the chunk is found for the right questions.

Quality and limitations

Q17. How do you evaluate a retrieval system? Build a test set of realistic questions, each with the document it should find. Measure how often the correct document is ranked first (top-1 accuracy) or appears in the top k. Re-run the test after every change to chunking, documents or models.

Q18. Why might semantic search return the wrong document, and how do you improve it? Common causes: ambiguous questions, generic text that looks similar everywhere, chunks that lack context, or vague documents. Improvements: clearer and more specific documents, better chunking, adding titles and metadata, combining semantic and keyword search (hybrid search), and re-ranking the top results.

Q19. What are the limitations of embeddings? They capture general meaning, so words with two senses can mislead ("secure website" vs "security attack"). Very short texts give weak signals. They struggle with exact values such as error codes or IDs, where keyword search is better. And they only know what the model learned, not your organisation's private terms unless they appear in your documents.

Q20. What is hybrid search? Combining keyword search and semantic search, then merging the results. It catches exact terms (error codes, hostnames) and paraphrased meaning, and usually works better than either one alone.

Big picture

Q21. Where does semantic search fit in RAG? It is the retrieval step. The question is embedded, the vector database returns the most relevant chunks, and those chunks are added to the prompt so the chat model answers from them instead of guessing.

Q22. Why not just send all documents to the LLM? They will not fit in the context window, it would be slow and expensive, and irrelevant text distracts the model and increases errors. Retrieval sends only the few most relevant pieces.

Q23. Describe a semantic search system in one minute. Documents are split into chunks, each chunk is turned into an embedding with an embedding model, and the text, embedding and metadata are stored in a vector database. At query time, the question is embedded with the same model, the database returns the closest chunks by similarity, and those chunks, with their sources, are shown to the user or passed to a chat model to generate an answer.